2026 Spring

Course ID: 1142-1765

AdvancedProgramming

01-Course Introduction

Advanced Programming

Python · C++ · Rust

Data Structures & Algorithms

Python | C++ | Rust

Dynamic Programming · Divide & Conquer · Greedy Algorithms · Arrays · Lists · Maps & Sets

Course Agenda

01

Language Deep Dives

Python · C++ · Rust advanced features

02

Data Structures

Arrays, Lists, Maps, Sets

03

Algorithms

DP · Divide & Conquer · Greedy

04

Language Comparison

Performance, safety, use cases

🏆 Top 10 Programming Languages
Source: TIOBE Index · November 2025
🥇 Python
23.37%
🥈 C++
10.03%
🥉 Java
9.45%
4 C
8.89%
5 C#
6.73%
6 JavaScript
3.79%
7 Go
2.27%
8 Rust 🦀
1.98% ↑
9 Swift
1.54%
10 TypeScript
1.42%
0%
5%
10%
15%
20%
23%
🦀 Rust reached its all-time high of #8 in Nov 2025 — memory safety & performance driving adoption

Python — Advanced Features

Comprehensions & Generators

List/dict/set comprehensions, generator expressions for memory-efficient iteration

Decorators & Context Managers

Higher-order functions, @property, @staticmethod, with statement patterns

Type Hints & Dataclasses

PEP 484 type annotations, @dataclass, Protocol for structural subtyping

Python

```python
# Generator + type hints + dataclass
from dataclasses import dataclass
from typing import Iterator

@dataclass
class Node:
    val: int
    children: list["Node"]

def traverse(root: Node) -> Iterator[int]:
    yield root.val
    for child in root.children:
        yield from traverse(child)

# Lazy evaluation — memory efficient
tree = Node(1, [Node(2, []), Node(3, [])])
result = list(traverse(tree))  # [1, 2, 3]
```

Key Concepts

- GIL & asyncio concurrency
- Magic methods (__dunder__)
- Metaclasses & ABCs
- functools: lru_cache, partial
- itertools: chain, islice, product
- Walrus operator :=

C++ — Advanced Features

Templates & Metaprogramming

Generic programming, SFINAE, constexpr if, fold expressions

Move Semantics & RAII

rvalue refs, std::move, unique_ptr, Rule of Five

Concurrency (C++17/20)

std::thread, mutex, async, coroutines (co_await)

C++

```
// Templates + move semantics + smart ptrs
template<typename T>
class Stack {
    std::vector<T> data_;
public:
    void push(T&& val) {          // rvalue ref
        data_.push_back(std::move(val));
    }
    std::optional<T> pop() {
        if (data_.empty()) return std::nullopt;
        T val = std::move(data_.back());
        data_.pop_back();
        return val;
    }
};
// RAII: auto cleanup, no manual delete
auto s = std::make_unique<Stack<int>>();
```

C++20 Highlights

- Concepts (type constraints)
- Ranges & views library
- Modules (replace headers)
- std::span — safe array views
- Coroutines: co_await/co_yield
- Three-way comparison <=>

🦀 Rust — Advanced Features

Ownership & Borrowing

Move semantics, lifetimes, borrow checker — zero-cost memory safety

Traits & Generics

Trait objects, impl Trait, associated types, trait bounds

Fearless Concurrency

Send/Sync traits, Arc<Mutex<T>>, async/await with Tokio

Rust

```rust
// Ownership + traits + generics
trait Summary {
    fn summarize(&self) -> String;
}
struct Article { title: String, content: String }

impl Summary for Article {
    fn summarize(&self) -> String {
        format!("{}: {}...", self.title,
            &self.content[..50])
    }
}
// Borrow checker ensures safety at compile time
fn notify(item: &impl Summary) {
    println!("Breaking: {}", item.summarize());
}
```

Why Rust?

- No null / no data races
- Zero-cost abstractions
- #8 TIOBE (Nov 2025) ↑
- 9× Most Admired (Stack Overflow)
- WebAssembly & embedded
- Linux kernel integration

Language Comparison

Feature	Python	C++	Rust
Speed	Interpreted / Slow	Near bare-metal	Near bare-metal
Memory Safety	GC / Runtime errors	Manual (error-prone)	Compile-time guaranteed
Learning Curve	★★☆☆☆ Easy	★★★★☆ Hard	★★★★★ Steep
Concurrency	asyncio / GIL limit	std::thread / complex	Fearless / Send+Sync
Use Cases	AI/ML, scripting, web	Systems, games, HPC	Systems, WebAssembly
Ecosystem	PyPI (500k+ pkgs)	vcpkg / Conan	crates.io (130k+ crates)
TIOBE 2025	#1 — 23.37%	#2 — 10.03%	#8 — 1.98% ↑

Data Structures

Arrays & Buffers

12	7	34	5	19	28	3	41
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]

O(1) random access · O(n) insert/delete · Contiguous memory

Python

```python
import array, numpy as np

# Static typed array
a = array.array('i', [1, 2, 3])

# NumPy array (vectorized)
arr = np.array([1, 2, 3, 4])
arr2 = arr * 2        # [2,4,6,8]
arr.reshape(2, 2)     # 2D matrix

# Slicing
arr[1:3]   # [2, 3]
arr[::-1]  # reversed
```

C++

```cpp
#include <array>
#include <vector>
#include <algorithm>

// Fixed-size array (stack)
std::array<int,4> a = {1,2,3,4};

// Dynamic array (heap)
std::vector<int> v = {5,6,7};
v.push_back(8);      // O(amortized 1)

// Sort + binary search
std::sort(v.begin(), v.end());
auto it = std::lower_bound(
    v.begin(), v.end(), 6);
```

Rust

```rust
// Fixed array (stack-allocated)
let arr: [i32; 4] = [1, 2, 3, 4];

// Dynamic Vec (heap)
let mut v: Vec<i32> = vec![5,6,7];
v.push(8);

// Slice — zero-copy view
let slice: &[i32] = &v[1..3];

// Sorting is safe, no UB
v.sort();
let pos = v.binary_search(&6);
```

Data Structures

Linked Lists

Head → **7** → **14** → **21** → null

O(1) insert/delete at head · O(n) random access · No contiguous memory needed

Python

```python
from collections import deque

# Python list = dynamic array
lst = [1, 2, 3, 4]
lst.insert(0, 0)   # O(n)
lst.pop()          # O(1)

# deque for efficient prepend
dq = deque([1, 2, 3])
dq.appendleft(0)   # O(1)
dq.rotate(1)

# Linked list via class
class Node:
    def __init__(self, val, nxt=None):
        self.val = val; self.next = nxt
```

C++

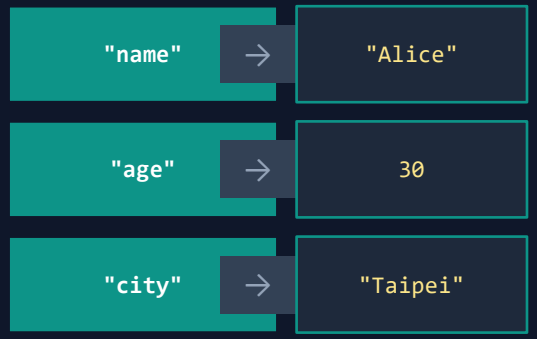
```cpp
#include <list>
#include <forward_list>

// Doubly linked list
std::list<int> lst = {1,2,3};
auto it = lst.begin();
lst.insert(it, 0);    // O(1)
lst.splice(it, lst);  // merge

// Singly linked
std::forward_list<int> fl;
fl.push_front(10);
fl.emplace_after(
  fl.begin(), 20);
```

Rust

```rust
use std::collections::LinkedList;
use std::collections::VecDeque;

// VecDeque: ring buffer
let mut dq = VecDeque::new();
dq.push_front(0);
dq.push_back(1);

// Linked list (rarely used)
let mut ll: LinkedList<i32>
    = LinkedList::new();
ll.push_back(1);

// Idiomatic: prefer Vec<T>
let v: Vec<i32> = vec![1,2,3];
```

Data Structures

Dictionary / Map / Set



Hash Map: O(1) avg lookup · O(n) worst | Set: O(1) membership test

Python

```python
# dict — ordered hash map
d = {"name": "Alice", "age": 30}
d["city"] = "Taipei"   # O(1)
d.get("name", "N/A")

# dict comprehension
squares = {x: x**2 for x in range(5)}

# set operations
a = {1, 2, 3}; b = {2, 3, 4}
a | b   # union
a & b   # intersection
a - b   # difference

from collections import Counter
```

C++

```cpp
#include <unordered_map>
#include <unordered_set>
#include <map>   // ordered (tree)

// Hash map: O(1) avg
std::unordered_map<string,int> um;
um["Alice"] = 30;
um.count("Alice"); // exists check

// Ordered map (red-black tree O(logn))
std::map<string,int> om;
om["beta"] = 2; om["alpha"] = 1;

// Set
std::unordered_set<int> s{1,2,3};
```

Rust

```rust
use std::collections::{
    HashMap, HashSet, BTreeMap};

let mut map: HashMap<&str, i32>
    = HashMap::new();
map.insert("Alice", 30);
map.entry("Bob").or_insert(25);

// BTreeMap: sorted (O log n)
let mut bm: BTreeMap<&str, i32>
    = BTreeMap::new();

let mut set: HashSet<i32>
    = HashSet::from([1, 2, 3]);
set.contains(&2);  // true
```

Dynamic Programming

Memoization

Top-down: cache recursive results to avoid recomputation

Tabulation

Bottom-up: fill table iteratively, use previously computed values

Optimal Substructure

Optimal solution built from optimal solutions to subproblems

Python

```python
# Python: Fibonacci DP (memoization vs tabulation)
from functools import lru_cache

# ① Top-down (memoization)
@lru_cache(maxsize=None)
def fib_memo(n: int) -> int:
    return n if n <= 1 else fib_memo(n-1) + fib_memo(n-2)

# ② Bottom-up (tabulation) — O(n) time, O(1) space
def fib_tab(n: int) -> int:
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a

# ③ 0/1 Knapsack — classic DP
def knapsack(weights, values, capacity):
    n = len(weights)
    dp = [[0] * (capacity+1) for _ in range(n+1)]
    for i in range(1, n+1):
        for w in range(capacity+1):
```

Classic DP Problems

- Fibonacci / Climbing Stairs
- 0/1 Knapsack
- Longest Common Subsequence
- Coin Change
- Matrix Chain Multiplication
- Edit Distance (Levenshtein)
- Floyd-Warshall (shortest paths)

⚔️ Divide & Conquer

1 Divide

Split problem into smaller sub-problems of same type

2 Conquer

Recursively solve each sub-problem (base case: trivial)

3 Combine

Merge sub-problem results into the final solution

C++

```cpp
// C++ Merge Sort — classic D&C, O(n log n)
#include <vector>
using namespace std;

void merge(vector<int>& arr, int l, int m, int r) {
    vector<int> tmp(arr.begin()+l, arr.begin()+m+1);
    int i = 0, j = m+1, k = l;
    while (i < (int)tmp.size() && j <= r)
        arr[k++] = tmp[i] <= arr[j] ? tmp[i++] : arr[j++];
    while (i < (int)tmp.size()) arr[k++] = tmp[i++];
}

void mergeSort(vector<int>& arr, int l, int r) {
    if (l >= r) return;          // base case
    int m = l + (r - l) / 2;
    mergeSort(arr, l, m);        // conquer left
    mergeSort(arr, m+1, r);      // conquer right
    merge(arr, l, m, r);         // combine
}
```

D&C Algorithms

- Merge Sort — O(n log n)
- Quick Sort — O(n log n) avg
- Binary Search — O(log n)
- Strassen Matrix Multiply
- Closest Pair of Points
- Fast Fourier Transform (FFT)
- Karatsuba Multiplication

Greedy Algorithms

✅ Greedy Choice

At each step, pick the locally optimal choice (best immediate gain)

🧩 Optimal Substructure

Greedy choice + optimal sub-solution = global optimal (when provable)

⚠️ When it Fails

Greedy ≠ always globally optimal (use DP instead for 0/1 Knapsack)

Python

```python
# Python: Greedy examples

# ① Activity Selection (Interval Scheduling)
def activity_selection(activities):
    # Sort by finish time (greedy choice)
    acts = sorted(activities, key=lambda x: x[1])
    selected = [acts[0]]
    for start, end in acts[1:]:
        if start >= selected[-1][1]:
            selected.append((start, end))
    return selected

# ② Fractional Knapsack (greedy works here!)
def fractional_knapsack(items, capacity):
    # Sort by value/weight ratio
    items.sort(key=lambda x: x[1]/x[0], reverse=True)
    total, remaining = 0, capacity
    for weight, value in items:
        if remaining == 0: break
        take = min(weight, remaining)
```

Greedy Problems

- Activity Selection
- Fractional Knapsack
- Huffman Encoding
- Dijkstra's Shortest Path
- Kruskal's / Prim's MST
- Job Scheduling
- Interval Scheduling Maximization

Algorithm Strategy Comparison

Property	🧮 Dynamic Programming	⚔️ Divide & Conquer	💰 Greedy
Approach	Overlapping subproblems + cache	Non-overlapping sub-problems	Local optimal at each step
Guarantee	Global optimum (always)	Correct by construction	Global optimum (when provable)
Complexity	Often $O(n^2)$ or $O(n \cdot W)$	$O(n \log n)$ typical	$O(n \log n)$ or $O(n)$
Space	$O(n)$ to $O(n \cdot W)$ table	$O(\log n)$ call stack	$O(1)$ to $O(n)$
When to Use	Overlapping subproblems	Independent sub-problems	Exchange argument works
Classic Example	0/1 Knapsack, LCS	Merge Sort, Quick Sort	Fractional Knapsack, Dijkstra

Key Takeaways

Python

Best for rapid development, AI/ML, scripting. Use generators, decorators, and type hints for clean, efficient code.

C++

Maximum performance with templates, move semantics, and RAII. Modern C++20 brings concepts, ranges, and coroutines.

Rust

Memory safety without GC. Ownership model eliminates entire bug classes. Growing fast — now #8 TIOBE.

Dynamic Programming

Break overlapping subproblems into table/memo. Always ask: can this be decomposed with caching?

Divide & Conquer

Split → solve independently → merge. Key for O(n log n) sorting and tree/graph algorithms.

Greedy

Fastest approach when exchange argument holds. Verify with counter-example before committing.

Master the fundamentals — the language is just a tool, the thinking is the skill.

Coding Tools

for Modern Programming Practice

VS Code

Local IDE

Extensions, debugging, Git

CodeChef IDE

Online · C++20

Competitive programming

Rust Playground

play.rust-lang.org

Browser-based Rust IDE

Google Colab

Notebooks · Cloud

Python + GPU/TPU access

Python · C++20 · Rust — Local, Online, and Cloud Environments

Tool Comparison Overview

Feature	💙 VS Code	🏆 CodeChef IDE	🦀 Rust Playground	📓 Google Colab
Type	Local IDE	Online IDE	Online REPL	Cloud Notebook
Languages	Any (extensible)	C++, Python, Java	Rust only	Python (+ R, Julia)
C++ Version	C++23 (clang)	C++20 ✅	N/A	g++ via terminal
Python	Full support	Python 3	N/A	Full + AI assist
Rust	rust-analyzer	Limited	Full (stable/nightly)	cargo install
GPU/TPU	Local GPU	No	No	Free T4 GPU ✅
Collaboration	Live Share	No	Share URL	Real-time sharing
Offline	Yes ✅	No	No	No
Cost	Free	Free	Free	Free (paid tiers)

Visual Studio Code

Free

Open Source

The #1 Code Editor — Free, Open Source, Cross-Platform

Extension Marketplace

50k+ extensions for every language & tool

Built-in Debugger

Breakpoints, watch vars, call stack for all languages

GitHub Copilot / AI

IntelliSense, AI completions, inline chat

Git Integration

Stage, commit, diff, branches without leaving editor

Integrated Terminal

Run Python, compile C++, cargo build — in-editor

Remote Development

SSH, WSL, Dev Containers — code anywhere

Essential Extensions for Python · C++ · Rust

Python

Microsoft

IntelliSense, Pylance, debugger

C/C++

Microsoft

IntelliSense, clang-format, debugger

rust-analyzer

rust-lang

Completions, inlay hints, cargo

GitHub Copilot

GitHub

AI pair programmer

Code Runner

Jun Han

Run snippets with one click

💙 VS Code — Setup & Workflow for Python, C++, Rust

Python

1. Install Python extension

2. Select interpreter: Ctrl+Shift+P

3. Create .venv virtual environment

4. Run: ▶ button or F5

Python

```python
# hello.py
import sys
print(f"Python {sys.version}")

# Install packages
# pip install numpy pandas

def fibonacci(n: int) -> int:
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

C++20

1. Install C/C++ + CMake Tools

2. Set compiler: clang++ or g++

3. tasks.json: add -std=c++20 flag

4. F5 to build & debug

C++20

```cpp
// main.cpp  (-std=c++20)
#include <print>  // C++23
#include <ranges>
#include <vector>

int main() {
    std::vector<int> v{1,2,3,4,5};
    // Ranges (C++20)
    auto evens = v | std::views::filter(
        [](int x){ return x%2==0; });
    std::print("Evens: ");
    for (auto e : evens)
        std::print("{} ", e);
}
```

Rust

1. Install rust-analyzer extension

2. rustup: install toolchain

3. cargo new my_project

4. cargo run / cargo test

Rust

```rust
// src/main.rs
fn main() {
    let numbers = vec![1, 2, 3, 4, 5];

    // Functional style
    let sum: i32 = numbers
        .iter()
        .filter(|&&x| x % 2 == 0)
        .sum();

    println!("Sum of evens: {}", sum);
    // cargo run — builds & executes
}
```

CodeChef IDE

codechef.com/ide — Supports C++20, Python 3, Java, and 50+ languages

C++20 ✅

Free

C++20 Features Available on CodeChef

Concepts

```
template<typename T>
  requires Integral<T>
```

Ranges & Views

```
v | views::filter(...)
  | views::transform(...)
```

Coroutines

```
co_await, co_yield,
  co_return
```

Modules

```
import std.core;
  (replaces #include)
```

Three-way <=>

```
auto cmp = a <=> b;
  std::strong_ordering
```

std::span

```
void f(std::span<int> s)
  // safe array view
```

C++20

```
// C++20 on CodeChef IDE — Concepts + Ranges
#include <iostream>
#include <vector>
#include <ranges>
#include <algorithm>
#include <concepts>
using namespace std;

// C++20 Concept
template<typename T>
concept Numeric = integral<T> || floating_point<T>;

// C++20 Constrained template
template<Numeric T>
T sum_of_squares(vector<T>& v) {
    auto squares = v | views::transform(
        [](T x) { return x * x; });
    T total = 0;
    for (auto sq : squares) total += sq;
    return total;
}

int main() {
    vector<int> nums = {1, 2, 3, 4, 5};

    // C++20 Ranges pipeline
    auto result = nums
        | views::filter([](int x){ return x%2!=0; })
        | views::transform([](int x){ return x*x; });
```

🏆 CodeChef IDE — Interface & How to Use

🌐 https://www.codechef.com/ide

⚙ Settings Panel

Language:	C++ 17 / C++ 20 ← USE THIS
Theme:	Dark / Light
Font Size:	14px (adjustable)
Input:	Custom stdin pane
Output:	stdout + stderr
Time limit:	Shown per problem

📄 main.cpp

▶ Run

```cpp
#include <bits/stdc++.h>
using namespace std;

// C++20: ranges & concepts
int main() {
    int n; cin >> n;
    vector<int> a(n);
    for (auto& x : a) cin >> x;

    // Sort descending (C++20 ranges)
    ranges::sort(a, greater<int>());

    for (int x : a)
        cout << x << " ";
    return 0;
}
```

Input: 5 3 1 4 1 5 Output: 5 4 3 1 1

💡 Pro Tips

- Select C++20 in language dropdown before writing
- #include <bits/stdc++.h> includes everything
- Use Custom Input box for test cases
- Ctrl+Enter = Run shortcut
- Press Submit to check against judge
- Great for LeetCode-style practice too
- No install needed — browser only

Rust Playground

play.rust-lang.org — Official browser-based Rust IDE by the Rust team

Stable

Nightly

Stable + Nightly

Switch between stable and nightly Rust toolchains instantly

Edition Selector

Choose Rust 2015, 2018, 2021, or 2024 editions

Tools: fmt/clippy

Format code (rustfmt) or run lints (clippy) in browser

Shareable URLs

Every snippet gets a unique URL for easy sharing

MIR / ASM View

See MIR, HIR, LLVM IR, or Assembly output

Crate Support

Limited crates available: rand, serde, itertools, etc.

Rust

```rust
// play.rust-lang.org — Ownership & Iterators
fn main() {
    // Ownership demo
    let s1 = String::from("hello");
    let s2 = s1.clone();  // deep copy
    println!("{} {}", s1, s2);  // both valid!

    // Pattern matching
    let opt: Option<i32> = Some(42);
    match opt {
        Some(n) if n > 0 => println!("positive: {n}"),
        Some(n) => println!("non-positive: {n}"),
        None => println!("nothing"),
    }

    // Functional iterator chains
```

🌙 Nightly-Only Features (select in Playground)

`#![feature(let_chains)]`

if let A = x && let B = y { }

`#![feature(generators)]`

yield-based generators

`#![feature(generic_const_exprs)]`

const generics expressions

`#![feature(async_fn_in_trait)]`

async in trait definitions

`#![feature(specialization)]`

specialize trait impls

Google Colab

colab.research.google.com — Cloud Jupyter Notebooks with free GPU/TPU access

Free GPU

Jupyter

Python Notebooks

Markdown + code cells, inline plots, variable explorer

Free T4 GPU / TPU

Train ML models free — CUDA, cuDNN pre-installed

Pre-installed Libs

numpy, pandas, matplotlib, scikit-learn, TensorFlow, PyTorch

Google Drive Mount

Access your Drive files directly in notebooks

Gemini AI Assist

AI code completion and cell explanation built-in

Share & Collaborate

Share link like Google Docs — real-time collaboration

Python

```python
# Google Colab — Python ML example
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Generate sample data
np.random.seed(42)
X = np.random.randn(100, 1) * 2
y = 3 * X.squeeze() + np.random.randn(100)

# Train model
model = LinearRegression()
model.fit(X, y)
print(f"Coefficient: {model.coef_[0]:.2f}")
print(f"Intercept:   {model.intercept_:.2f}")
```

Enable Free GPU in Colab

Runtime → Change runtime type

Select T4 GPU from dropdown

import torch

torch.cuda.is_available() → True

!nvidia-smi

Check GPU model & memory

device = torch.device('cuda')

Move tensors to GPU

model.to(device)

Train at GPU speed 🚀

Google Colab — Running C++ and Rust in Notebooks

Colab is not just for Python — use shell cells to compile C++ and install Rust!

Running C++20 in Colab

Shell + C++20

```
# In a Colab code cell (note the ! prefix)
# Check compiler version
!g++ --version
# g++ (Ubuntu) 11.4.0 — supports C++20

# Write C++20 file from Python cell
%%writefile solution.cpp
#include <bits/stdc++.h>
#include <ranges>
using namespace std;

int main() {
    // C++20 ranges
    vector<int> v{5,3,1,4,2};
    ranges::sort(v);

    // C++20 format (if available)
    for (auto x : v | views::take(3))
        cout << x << " ";
    cout << endl;
    return 0;
}

# Compile with C++20 standard
!g++ -std=c++20 -O2 solution.cpp -o solution
```

Running Rust in Colab

Shell + Rust

```
# Install Rust (one-time per session)
!curl --proto '=https' --tlsv1.2 \
  -sSf https://sh.rustup.rs | sh -s -- -y
import os
os.environ["PATH"] += ":/root/.cargo/bin"

# Verify installation
!rustc --version
# rustc 1.xx.0 (stable)

# Write Rust file
%%writefile main.rs
fn fibonacci(n: u64) -> u64 {
    match n {
        0 | 1 => n,
        _ => fibonacci(n-1) + fibonacci(n-2),
    }
}

fn main() {
    for i in 0..10 {
        println!("fib({}) = {}", i, fibonacci(i));
    }
}

# Compile and run
```

Which Tool Should I Use? — Decision Guide

🏆 Competitive Programming & LeetCode Practice

👉 CodeChef IDE

- CodeChef: C++20, quick runs, no setup
- VS Code: local, debug, full project

🐍 Python Data Science & Machine Learning

👉 Google Colab

- Colab: free GPU, pre-installed libs
- VS Code: local control, git workflow

🦀 Learning Rust & Quick Experiments

👉 Rust Playground

- Playground: instant, shareable, no install
- VS Code: full projects, cargo, crates.io

👩‍🏫 Teaching / Demos & No Installation

👉 Browser Tools

- All browser-based, no student setup needed
- Share URL = instant code sharing

🔬 Large Projects & Professional Dev

👉 VS Code

- Debugger, extensions, Git, terminal
- Full language server support for all 3 langs

⚡ Just Testing C++20 Features

👉 CodeChef IDE

- C++20 support confirmed ✅
- No compile flags needed, paste & run

Same Problem, 4 Tools — FizzBuzz + Sum (see the workflow difference)

💙 VS Code + Python

Local · run in terminal

```python
# fizzbuzz.py (VS Code)
# Run: python fizzbuzz.py
import sys

def fizzbuzz(n: int) -> str:
    match (n%3==0, n%5==0):
        case (True, True):
return "FizzBuzz"
        case (True, False):
return "Fizz"
        case (False, True):
return "Buzz"
        case _: return str(n)

total = sum(int(fizzbuzz(i)
or i)
            if
fizzbuzz(i).isdigit()
            else 0
            for i in range(1,
101))
print([fizzbuzz(i) for i in
range(1,16)])
```

🏆 CodeChef C++20

Online · C++20 standard

```cpp
// CodeChef IDE — C++20
#include <bits/stdc++.h>
using namespace std;

// C++20 concept
template<integral T>
string fizzbuzz(T n) {
    if (n%15==0) return
"FizzBuzz";
    if (n%3==0)  return
"Fizz";
    if (n%5==0)  return
"Buzz";
    return to_string(n);
}
int main() {
    // C++20 ranges
    for (int i :
views::iota(1,16))
        cout << fizzbuzz(i)
<< "\n";
    return 0;
}
```

🧶 Rust Playground

play.rust-lang.org

```rust
// Rust Playground — pattern
match
fn fizzbuzz(n: u32) -> String
{
    match (n % 3, n % 5) {
        (0, 0) =>
"FizzBuzz".to_string(),
        (0, _) =>
"Fizz".to_string(),
        (_, 0) =>
"Buzz".to_string(),
        _      =>
n.to_string(),
    }
}
fn main() {
    // Functional style
    let results: Vec<String>
=
        (1..=15)
        .map(fizzbuzz)
        .collect();
    println!("{}",
results.join(", "));
}
```

📓 Google Colab

Notebook · inline output

```python
# Google Colab cell
# Output displayed inline
below cell

def fizzbuzz(n):
    if n % 15 == 0: return
"FizzBuzz"
    if n % 3  == 0: return
"Fizz"
    if n % 5  == 0: return
"Buzz"
    return str(n)

results = [fizzbuzz(i) for i
in range(1,16)]
print(results)

# Colab bonus: visualize!
import pandas as pd
df = pd.DataFrame({"n":
range(1,21),
    "result": [fizzbuzz(i)
for i in range(1,21)]})
df.head(10)  # renders as
table
```

⌨️ Keyboard Shortcuts Cheat Sheet

💙 VS Code

`Ctrl+P`	Quick file open
`Ctrl+Shift+P`	Command palette
`F5`	Start debugging
`Ctrl+` `	Toggle terminal
`Ctrl+/`	Toggle comment
`Alt+↑/↓`	Move line up/down
`Ctrl+D`	Select next match
`Ctrl+Shift+L`	Select all matches

🏆 CodeChef

`Ctrl+Enter`	Run code
`Ctrl+Z`	Undo
`Ctrl+Shift+Z`	Redo
`Tab`	Indent code
`Ctrl+A`	Select all
`Ctrl+C/V`	Copy / Paste
`F11`	Fullscreen editor
`Ctrl+S`	Save (download)

🦀 Rust Playground

`Ctrl+Enter`	Run code
`Alt+F`	Format (rustfmt)
`Alt+C`	Clippy lints
`Ctrl+Z`	Undo
`Share button`	Get permalink URL
`Ctrl+/`	Toggle comment
`Tools menu`	MIR / ASM view
`Stable/Nightly`	Switch channel

📓 Google Colab

`Ctrl+Enter`	Run cell
`Shift+Enter`	Run + next cell
`Alt+Enter`	Run + insert below
`Ctrl+M B`	Insert cell below
`Ctrl+M D`	Delete cell
`Ctrl+M M`	Cell to Markdown
`Ctrl+M Y`	Cell to Code
`Ctrl+F9`	Run all cells

Summary & Quick Reference

VS Code

`code.visualstudio.com`

- Best for full projects & professional dev
- **Python + C++ + Rust all in one IDE**
- Debugger, Git, Copilot AI, Extensions
- Use rust-analyzer for Rust LSP
- tasks.json: set -std=c++20 for C++20

CodeChef IDE

`codechef.com/ide`

- Best for competitive programming practice
- **C++20 fully supported ✅ — select in dropdown**
- No setup: paste code, click Run
- #include <bits/stdc++.h> works perfectly
- Python 3, Java, C, 50+ languages too

Rust Playground

`play.rust-lang.org`

- Best for learning Rust without any install
- **Stable + Nightly channels available**
- Share URL = instant code sharing
- Run rustfmt & clippy in the browser
- View MIR / Assembly output

Google Colab

`colab.research.google.com`

- Best for Python ML/AI with free GPU
- **Run C++ with !g++ -std=c++20 shell cells**
- Install Rust via rustup in one cell
- Google Drive integration & sharing
- Gemini AI assistance built-in

Use CodeChef for C++20 · Rust Playground for Rust · Colab for Python AI/ML · VS Code for everything else 🚀

LeetCode Practice

Master Programming Skills with Python · C++ · Rust

2500+	50+	10M+
Problems	Companies	Users

Difficulty Tiers

- Easy
- Medium
- Hard

Arrays · Strings · Trees · Graphs · DP · Greedy · Divide & Conquer · Two Pointers · Sliding Window

OVERVIEW

Why LeetCode & Problem-Solving Framework

🎯 Interview Prep

Used by FAANG & top companies worldwide for technical screening

🧠 Skill Building

Systematic way to practice DS&A across multiple languages

📈 Progress Tracking

Difficulty ratings, acceptance rates, and topic tags guide learning

The UMPIRE Problem-Solving Framework

U Understand

Read carefully, identify inputs/outputs, ask clarifying questions

M Match

Recognize pattern — is this DP? Two Pointers? BFS? Hash Map?

P Plan

Pseudocode the approach, estimate time/space complexity

I Implement

Code cleanly with meaningful variable names

R Review

Trace through examples, check edge cases (empty, single element)

E Evaluate

Analyze Big-O, look for optimizations, compare approaches

LANGUAGE TIPS

LeetCode Language-Specific Tricks & Shortcuts

Python

- use collections.defaultdict, Counter, deque
- heapq for min-heap (negate for max)
- bisect.bisect_left for binary search
- @lru_cache(None) for instant memoization
- zip(*matrix) to transpose a 2D list
- enumerate(), zip() avoid index management

C++

- unordered_map/set for O(1) avg lookup
- priority_queue<> for heap (max default)
- auto& avoids copying in range-for loops
- emplace_back faster than push_back
- lower_bound / upper_bound for sorted arrays
- INT_MAX / INT_MIN for sentinel values

Rust

- BinaryHeap<Reverse<T>> for min-heap
- .iter().enumerate() for indexed loops
- HashMap::entry().or_insert() pattern
- Use i32 (not usize) to avoid subtract overflow
- .windows(k) and .chunks(k) for sliding ops
- Compiler forces you to handle all edge cases!

Algorithms

Common LeetCode Patterns & When to Use Them

Pattern	Recognition Signal	LeetCode Examples
Two Pointers	Sorted array, pairs, palindrome	#167 Two Sum II, #125 Valid Palindrome
Sliding Window	Subarray/substring of size k, max/min	#3 Longest Substring, #209 Min Size Subarray
BFS / Level-order	Shortest path, tree levels, islands	#102 Level Order, #200 Number of Islands
DFS / Backtracking	All combinations, permutations, paths	#46 Permutations, #79 Word Search
Binary Search	Sorted input, minimize/maximize answer	#704 Binary Search, #33 Search Rotated Array
Dynamic Programming	Optimal substructure, overlapping sub-problems	#70 Climbing Stairs, #322 Coin Change
Greedy	Local optimal → global optimal, intervals	#56 Merge Intervals, #435 Non-overlapping
Divide & Conquer	Halving problem, merge-type operations	#912 Sort Array, #148 Sort List

ROADMAP

LeetCode Study Plan — 12-Week Roadmap

Wk 1–2 · Easy

Arrays & Strings

#1 Two Sum · #26 Remove Dups · #88 Merge Sorted · #125 Valid Palindrome

Wk 3–4 · Easy

Hash Map & Stack

#20 Valid Parens · #49 Group Anagrams · #155 Min Stack · #217 Contains Dup

Wk 5–6 · Medium

Binary Search & Sort

#33 Search Rotated · #56 Merge Intervals · #704 Binary Search · #912 Sort

Wk 7–8 · Medium

Trees & Graphs (BFS/DFS)

#102 Level Order · #104 Max Depth · #200 Islands · #207 Course Schedule

Wk 9–10 · Medium

Dynamic Programming

#70 Climbing Stairs · #322 Coin Change · #300 LIS · #1143 LCS

Wk 11–12 · Hard

Hard & Mixed Practice

#23 Merge K Lists · #42 Trapping Rain · #124 Path Sum · #295 Median

Key Takeaways

🗺️ Use UMPIRE

Always understand the problem, match the pattern, plan before coding. Don't rush to type.

🔁 Learn Patterns

~14 core patterns cover 90% of LeetCode. Master each one before moving to the next.

⚡ C++: Performance

Use C++ when TLE strikes. STL (unordered_map, priority_queue) gives fine control.

🐍 Python: Prototype

Use Python for quick solutions. defaultdict, heapq, Counter, and @lru_cache save time.

🦀 Rust: Correctness

Rust's borrow checker catches edge cases. Great for practicing memory-safe algorithms.

📈 Track Progress

Do 1 Easy daily, 2 Mediums weekly, 1 Hard weekly. Consistency > cramming sessions.

"Every expert was once a beginner. The difference is they didn't stop." — Practice daily, one problem at a time. 🚀

LC #1 | EASY | Array / Hash Map

Two Sum

Problem:

Given an array of integers nums and an integer target, return indices of the two numbers that add up to target. Assume each input has exactly one solution.

Key Insight

For each number x, check if (target - x) already exists in a hash map. Store {value → index} as you iterate. One pass = O(n) vs O(n^2) brute force.

Python

```python
def twoSum(nums, target):
    seen = {}  # val → index
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []

# Example:
# nums=[2,7,11,15], target=9
# Output: [0,1]
```

C++

```cpp
vector<int> twoSum(
    vector<int>& nums, int target) {
  unordered_map<int,int> seen;
  for (int i = 0; i < nums.size(); i++){
    int comp = target - nums[i];
    if (seen.count(comp))
      return {seen[comp], i};
    seen[nums[i]] = i;
  }
  return {};
}
// Time: O(n) | Space: O(n)
```

LC #66 | EASY | Array / Math

Plus One

Problem:

You are given a large integer represented as an integer array digits, where each digit is a single digit. Add one to the integer and return the resulting array.

Key Insight

Traverse from right to left. If digit < 9, increment and return. If digit == 9, set to 0 and carry. If all 9s (e.g. [9,9,9]), prepend 1 to get [1,0,0,0].

Python

```python
def plusOne(digits: list) -> list:
    for i in range(len(digits)-1, -1, -1):
        if digits[i] < 9:
            digits[i] += 1
            return digits
        digits[i] = 0
    # All digits were 9
    return [1] + digits

# [1,2,9] → [1,3,0]
# [9,9,9] → [1,0,0,0]
# Time: O(n) | Space: O(1)
```

C++

```cpp
vector<int> plusOne(vector<int>& digits) {
    for (int i = digits.size()-1; i >= 0; i--){
        if (digits[i] < 9) {
            digits[i]++;
            return digits;
        }
        digits[i] = 0;
    }
    // All digits were 9 → prepend 1
    digits.insert(digits.begin(), 1);
    return digits;
}
// Time: O(n) | Space: O(1)
```

LC #20 | EASY | Stack / String

Valid Parentheses

Problem:

Given a string s containing only '(', ')', '{', '}', '[', ']', determine if the input string is valid. Brackets must close in correct order.

Key Insight

Push open brackets onto a stack. When a closing bracket is encountered, pop the stack and verify it matches. If stack is empty at the end → valid.

Python

```python
def isValid(s: str) -> bool:
    stack = []
    mapping = {')':'(', '}':'{', ']':'['}
    for ch in s:
        if ch in mapping:
            top = stack.pop() if stack else '#'
            if mapping[ch] != top:
                return False
        else:
            stack.append(ch)
    return not stack

# "({[]})" → push (, {, [
# then pop & match on ], }, ) → True
```

C++

```cpp
bool isValid(string s) {
    stack<char> st;
    for (char c : s) {
        if (c=='(' || c=='[' || c=='{')
            st.push(c);
        else {
            if (st.empty()) return false;
            char t = st.top(); st.pop();
            if ((c==')' && t!='(') ||
                (c==']' && t!='[') ||
                (c=='}' && t!='{'))
                return false;
        }
    }
    return st.empty();
}
// Time: O(n) | Space: O(n)
```

LC #21 | EASY | Linked List

Merge Two Sorted Lists

Problem:

You are given the heads of two sorted linked lists list1 and list2. Merge the two lists in sorted order and return the head of the merged list.

Key Insight

Use a dummy head node. Compare list1 and list2 values, advance the pointer of the smaller value, appending to the result. At the end attach the remaining list.

Python

```python
def mergeTwoLists(l1, l2):
    dummy = ListNode(0)
    cur = dummy
    while l1 and l2:
        if l1.val <= l2.val:
            cur.next = l1
            l1 = l1.next
        else:
            cur.next = l2
            l2 = l2.next
        cur = cur.next
    cur.next = l1 or l2
    return dummy.next

# Time: O(n+m) | Space: O(1)
```

C++

```cpp
ListNode* mergeTwoLists(
    ListNode* l1, ListNode* l2) {
  ListNode dummy(0);
  ListNode* cur = &dummy;
  while (l1 && l2) {
    if (l1->val <= l2->val) {
      cur->next = l1; l1 = l1->next;
    } else {
      cur->next = l2; l2 = l2->next;
    }
    cur = cur->next;
  }
  cur->next = l1 ? l1 : l2;
  return dummy.next;
}
// Time: O(n+m) | Space: O(1)
```

LC #121 | EASY | Array / Greedy

Best Time to Buy & Sell Stock

Problem:

Given an array prices where prices[i] is the price on day i, return the maximum profit from buying on one day and selling on a later day. Return 0 if no profit is possible.

Key Insight

Track the minimum price seen so far and the maximum profit. For each price, compute profit = price - minPrice and update maxProfit. Single pass O(n).

Python

```python
def maxProfit(prices: list) -> int:
    min_price = float('inf')
    max_profit = 0
    for price in prices:
        if price < min_price:
            min_price = price
        elif price - min_price > max_profit:
            max_profit = price - min_price
    return max_profit

# [7,1,5,3,6,4] → buy@1 sell@6 = 5
# Time: O(n) | Space: O(1)
```

C++

```cpp
int maxProfit(vector<int>& prices) {
    int minPrice = INT_MAX, maxProfit = 0;
    for (int price : prices) {
        if (price < minPrice)
            minPrice = price;
        else if (price - minPrice > maxProfit)
            maxProfit = price - minPrice;
    }
    return maxProfit;
}

// [7,1,5,3,6,4] → 5
// [7,6,4,3,1]   → 0 (no profit)
// Time: O(n) | Space: O(1)
```

LC #125 | EASY | Two Pointers / String

Valid Palindrome

Problem:

A phrase is a palindrome if, after converting all uppercase to lowercase and removing all non-alphanumeric characters, it reads the same forward and backward.

Key Insight

Use two pointers — left and right. Skip non-alphanumeric characters. Compare characters (case-insensitive). If any mismatch, return False. If pointers cross, return True.

Python

```python
def isPalindrome(s: str) -> bool:
    l, r = 0, len(s) - 1
    while l < r:
        while l < r and not s[l].isalnum():
            l += 1
        while l < r and not s[r].isalnum():
            r -= 1
        if s[l].lower() != s[r].lower():
            return False
        l += 1; r -= 1
    return True

# "A man, a plan, a canal: Panama" → True
```

C++

```cpp
bool isPalindrome(string s) {
    int l = 0, r = s.size() - 1;
    while (l < r) {
        while (l<r && !isalnum(s[l])) l++;
        while (l<r && !isalnum(s[r])) r--;
        if (tolower(s[l]) != tolower(s[r]))
            return false;
        l++; r--;
    }
    return true;
}

// "race a car" → false
// Time: O(n) | Space: O(1)
```

Creating a

GitHub Repository

Remote Storage

Host code in the cloud

Collaboration

Work with any team

Version Control

Track every change

What is GitHub?

GitHub is a cloud-based platform for hosting Git repositories — enabling version control, code collaboration, and project management.

Version Control

Git tracks every change to your code. Revert to any previous state at any time. Nothing is ever truly lost.

Collaboration

Multiple developers can work on the same project simultaneously using branches and pull requests.

Cloud Backup

Your code is stored remotely on GitHub's servers. Access it anywhere, share it with anyone.

Project Management

Issues, milestones, wikis, and CI/CD pipelines — GitHub manages the full software lifecycle.

Key Concepts Before You Start

Understanding these terms will make your GitHub workflow much clearer.

Repository (Repo)

A project folder that stores all your files and the entire revision history. Think of it as a super-powered project directory.

Commit

A saved snapshot of your changes. Every commit has a unique ID and message describing what changed. Like a save point in a game.

Branch

An independent line of development. The default is called 'main'. Create branches to work on features without affecting the main code.

Clone

Downloading a full copy of a remote repository to your local machine so you can work on it offline.

Push

Uploading your local commits to the remote GitHub repository so others can see and pull your changes.

Pull Request (PR)

A request to merge changes from one branch into another. It enables code review and team discussion before merging.

1 Sign Up & Set Up Your Account

Go to github.com and register

Enter your details

Username, email address, and a strong password. Your username will be part of your public URL.

Verify your email

GitHub sends a confirmation email. Click the link to activate your account before proceeding.

Choose a plan

Free tier is sufficient for most students. Public repos are unlimited; private repos allowed on free plan.

Enable 2FA (recommended)

Two-factor authentication adds security. Use an authenticator app like Google Authenticator.

Your GitHub profile URL will be:

`github.com/your-username`

Username Tips

- Use lowercase letters and hyphens only
- Keep it professional — it appears on your resume
- Avoid numbers at the end (e.g., user123)
- Make it short and memorable

2 Create a New Repository

github.com/new

Create a new repository

Owner

your-username

Repository name *

my-first-repo

Description (optional)

My first GitHub project

Visibility

Public | Private

Repository Name

Must be unique within your account. Use lowercase, hyphens, no spaces. e.g. my-cool-project

Public vs Private

Public: anyone can view. Private: only you and invited collaborators. Students often use Public for portfolios.

Description

Optional but recommended. A brief description helps others understand your project's purpose.

Tip

Click the green '+' button (top-right on GitHub) or go to github.com/new to start creating a repo.

3 Initialize Repository Files

README.md

Highly Recommended

The front page of your repository. Written in Markdown, it describes what the project does, how to install it, how to use it, and how to contribute.

```
# My Project

A brief description.

## Installation
```bash
npm install
```
```

.gitignore

Recommended

Tells Git which files to NOT track (e.g., node_modules/, .env, __pycache__). GitHub offers templates for Python, Node.js, Java, etc.

```
# Python example
__pycache__/
*.pyc
.env
venv/
.DS_Store
```

LICENSE

Optional

Defines how others may use your code. MIT License is the most permissive and widely used. Required if you want others to legally reuse your work.

```
MIT License

Copyright (c) 2025
Your Name

Permission is hereby
granted, free of charge...
```

4 Clone the Repository to Your Computer

Copy the Clone URL from GitHub: `https://github.com/username/my-first-repo.git`

HTTPS vs SSH

HTTPS (Recommended for beginners)

`https://github.com/user/repo.git`

Use your GitHub username & Personal Access Token as password.

SSH (For advanced users)

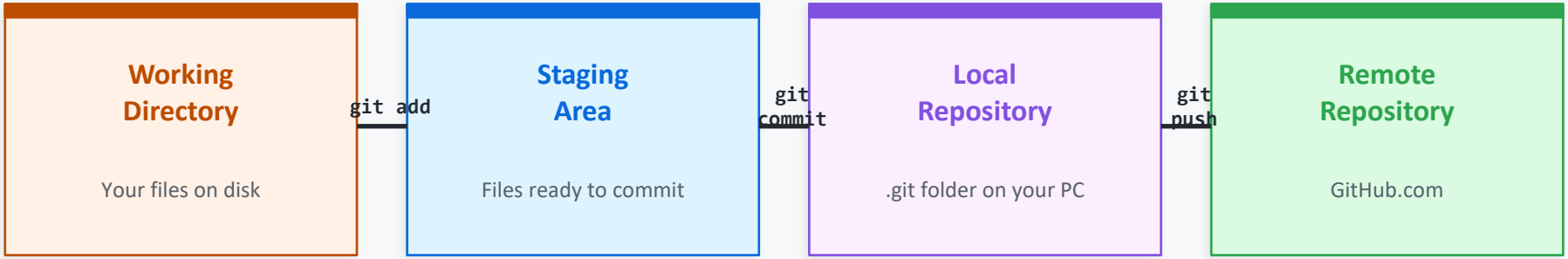
`git@github.com:user/repo.git`

Requires SSH key setup but no password prompt after configuration.

Terminal

```
$ git clone https://github.com/
    username/my-first-repo.git
  Cloning into 'my-first-repo'...
  remote: Enumerating objects: 3...
  Receiving objects: 100% (3/3)
$ cd my-first-repo
$ ls
  README.md  .gitignore  LICENSE
```

5 Add Files & Make Your First Commit



~ /my-first-repo

```
$ # Create a new file
$ echo 'print("Hello")' > hello.py
$ git status
  Untracked files: hello.py
$ git add hello.py
$ git commit -m "Add hello.py"
  [main abc1234] Add hello.py
```

`git status`

Shows which files are changed, staged, or untracked. Always run this first.

`git add <file>`

Stages a specific file. Use 'git add .' to stage all changed files at once.

`git commit -m "..."`

Saves staged changes to local repo with a descriptive message. Be descriptive!

6 Push to GitHub

Terminal

```
$ # Push to GitHub for the first time
$ git push -u origin main
  Enumerating objects: 5, done.
  Counting objects: 100% (5/5)
  Writing objects: 100% (3/3)
  To https://github.com/user/repo.git
     abc1234..def5678  main -> main

$ # Future pushes — just use:
$ git push

$ # To pull latest changes:
$ git pull origin main
```

`git push -u origin main`

First-time push
The -u flag sets the upstream tracking branch. After this, you can just type 'git push' for all future pushes.

`origin`

What is 'origin'?
A shorthand alias for your remote GitHub URL. Automatically set when you clone a repo.

`main`

What is 'main'?
The default branch name (formerly 'master'). All new GitHub repos now use 'main' as the default.

`git pull`

Stay up to date
Fetches and merges remote changes into your local branch. Run this before starting work each day.

>_ Essential Git Commands — Quick Reference

`git init`

Initialize a new local Git repo in current folder

`git clone <url>`

Clone (download) a remote repository

`git status`

Show changed / untracked / staged files

`git log --oneline`

Show compact commit history

`git add <file>`

Stage a specific file for the next commit

`git add .`

Stage ALL changed files at once

`git commit -m "message"`

Commit staged changes with a descriptive message

`git push`

Upload local commits to GitHub remote

`git pull`

Download + merge remote changes into local branch

`git branch <name>`

Create a new branch

`git checkout <branch>`

Switch to another branch

`git merge <branch>`

Merge a branch into the current branch

Complete Workflow at a Glance

1

Sign Up

`github.com`

2

New Repo

`github.com/new`

3

Add Files

`README, .gitignore`

4

Clone

`git clone <url>`

5

Edit & Stage

`git add .`

6

Commit

`git commit -m "..."`

7

Push

`git push`

8

Collaborate

`Pull Request`

Repeat steps 5-7 for every change. Collaborate using branches and pull requests.

GITHUB DESKTOP

A Friendlier Way to Use Git

GitHub Desktop is a free GUI app that lets you use Git and GitHub visually — no terminal commands needed.

Why Use GitHub Desktop?

- Visual diff viewer — see exactly what changed line by line
- No terminal needed — perfect for beginners and designers
- One-click commit, push, pull and branch switching
- Stage individual files or specific lines with checkboxes
- Visual merge conflict editor — no confusing markers
- Works on Windows and macOS — free forever

CLI vs GitHub Desktop

Task	Terminal	GitHub Desktop
Stage files	`git add .`	Check boxes
Commit	`git commit -m "..."`	Fill form + click
Push	`git push`	Click Push button
See changes	`git diff`	Visual side-by-side
Switch branch	`git checkout <b>`	Dropdown menu
Create branch	`git branch <n>`	New Branch button

GITHUB DESKTOP

Download, Install & Sign In

1 Download

desktop.github.com

Visit desktop.github.com and click Download. Available for Windows (.exe) and macOS (.dmg). The installer is around 100 MB.

2 Install

Run the installer

Windows: Run the .exe installer, no manual setup needed. macOS: Drag app to Applications folder. GitHub Desktop updates itself.

3 Sign In

github.com account

Open GitHub Desktop → File → Options → Accounts → Sign in to GitHub.com. Authorize the OAuth app in your browser.

4 Configure

Name & email

GitHub Desktop auto-fills your name and email from your GitHub account. This info appears on every commit you make.

5 Ready!

Clone or create

Welcome screen lets you: Clone a repo from GitHub, create a new local repo, or add an existing project folder on your computer.

Tip: GitHub Desktop also installs Git automatically if it is not already on your machine!

GITHUB DESKTOP

Daily Workflow in GitHub Desktop

Step-by-Step: Commit & Push

1. **Fetch / Pull first**
   Click Fetch origin at the top bar. Downloads any changes teammates pushed before you start editing.
2. **Edit your files**
   Make changes in VS Code or any editor. Switch back to GitHub Desktop when ready to save a snapshot.
3. **Review Changes tab**
   Left sidebar shows every modified file. Click any file to see a visual diff — green = added, red = removed.
4. **Stage (check) files**
   Check boxes next to files to include in this commit. Uncheck files you want to save for a future commit.
5. **Write commit message**
   Type a short summary in the bottom-left box (required). Add an optional longer description. Click Commit to main.
6. **Push to origin**
   Click the Push origin button in the top bar. Commits go live on GitHub.com — teammates can now pull them.

Key UI Areas to Know

Repository Bar (top)
Switch repos, branches, and push/fetch all from one top toolbar.

Changes Tab
Lists every modified file with color-coded diff. Check to stage.

History Tab
Shows every past commit — click any to see what files changed.

Branch Dropdown
Switch, create, or delete branches with no terminal needed.

Push / Fetch Button
Top-right button syncs your work — shows commits ahead/behind.

Open in Editor
Right-click a file to open it in VS Code or reveal in Finder/Explorer.

GitHub Desktop saves your credentials — no typing passwords or tokens on every push!

You're Ready to

Start Collaborating!

Create & Clone Repos

Commit Changes Locally

Push to GitHub

Next Steps: Branches, Pull Requests, Issues, GitHub Actions, and open-source contribution!

github.com · docs.github.com · git-scm.com

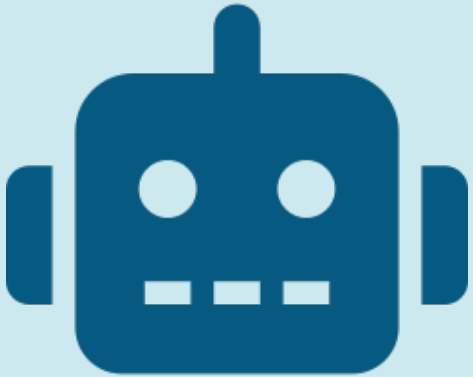
ADVANCED PROGRAMMING

GenAI Literacy

for Advanced Programming

Gemini · GitHub Copilot · Google Colab

Providence University | College of Computing and Informatics



AI-Powered · Critical Thinking · Responsible Use

The Three Core Tools

Each tool serves a distinct role — knowing when to use which is a key skill.

GitHub Copilot

IDE-Embedded Code Assistant

- Real-time code completion & refactoring
- Inline chat for function generation
- Works inside VS Code, JetBrains IDEs

Best for: Writing & iterating code fast

Google Gemini

Multimodal AI Assistant

- Architecture design & code explanation
- Multimodal: image + code analysis
- Conversational software development

Best for: Planning, design & multi-step reasoning

Google Colab

Cloud Execution + AI Integration

- Run Python in cloud with free GPU
- Built-in Gemini AI assistance
- Reproducible research & teaching demos

Best for: Experiments, ML, sharing notebooks

Why GenAI Literacy?

In advanced programming, GenAI is not just a tool — it's a core engineering workflow.

70%+

of developers

now use AI coding tools daily in production

3× faster

prototype speed

when using AI tools with good prompt skills

40% error rate

without review

AI-generated code has bugs if accepted blindly

Critical skill

in hiring

Employers now expect GenAI literacy in CS graduates

Advanced students must go beyond tool use — they must evaluate, audit, and integrate GenAI critically.

MOD 1

Prompt Engineering for Programmers

Core Concepts

Zero-shot Prompting

Ask directly — no examples needed. Good for simple tasks.

Few-shot Prompting

Provide 1–3 examples of expected input/output before your request.

Role Setting

Tell the AI who it is: "You are a senior Python developer..."

Constraint Specification

Define limits: format, length, library, error handling style.

❌ Weak Prompt

"write a function to sort data"

→ Ambiguous data type, no language, no edge cases.

✅ Strong Prompt

"You are a Python expert. Write a function that sorts a list of Patient objects by age (ascending), then by name alphabetically. Handle None values gracefully. Include type hints and docstring."

Lab: Compare AI outputs from weak vs. strong prompts for the same coding task — document the difference.

MOD 2

GitHub Copilot — Deep Dive

Capabilities

Tab Completion

Autocompletes functions, loops, and boilerplate as you type.

Inline Chat

Ask questions mid-code: "What does this function do?"

Test Generation

Generate unit tests automatically from function signatures.

Code Refactoring

Suggests cleaner, more Pythonic or idiomatic rewrites.

Known Limitations — Must Audit!

Hallucinated APIs

Generates plausible but non-existent function names or parameters.

Outdated Patterns

May suggest deprecated libraries or old syntax versions.

Security Gaps

Can introduce SQL injection, hardcoded secrets, or unsafe eval().

Logic Errors

Passes syntax checks but produces wrong results on edge cases.

Lab: Use Copilot to implement a sorting algorithm, then write test cases to find bugs in its output.

MOD 3

Google Gemini for Advanced Development

Architecture Design

Conversational development — iteratively design system architecture with Gemini, refine requirements, and generate scaffolding code.

Multimodal Analysis

Upload a medical image or diagram and ask: "Write Python code to process this type of image using OpenCV." Gemini connects vision to code.

Code Explanation

Paste complex code and ask Gemini to explain, identify bottlenecks, suggest optimizations, or translate to another language.

Python — Gemini API Call Example

```python
import google.generativeai as genai
genai.configure(api_key="YOUR_API_KEY")
model = genai.GenerativeModel("gemini-1.5-pro")
response = model.generate_content("Explain this Python function: " + my_code)
print(response.text)
```

GenAI Literacy Framework

Advanced courses target mid-to-high tier competencies — not just tool operation.

Advanced Programming Focus →

EVALUATE

Advanced

Verify, Audit, Ethical Judgment, IP & Privacy

INTEGRATE

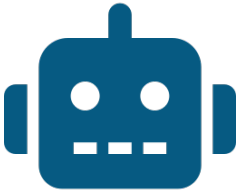
Intermediate

Tool Selection, Workflow Design, Multi-tool Pipelines

OPERATE

Foundation

Prompting, API Calls, IDE Use, Colab Notebooks



is not about using AI — it's about thinking with it.

Prompt Strategically

Evaluate Critically

Build Responsibly

Providence University | College of Computing and Informatics